



Curso: MEI+MCA+MMC
U.C.: Computação Paralela

Teste 9-Jan-2026
Duração: 2h

Parte I (35% - 70 pontos)

1. ¹⁰**Explique** as principais diferenças entre o paralelismo ao nível da instrução e o paralelismo baseado em fios de execução (*threads*). Indique quais os tipos comuns dependências que impedem a exploração desse paralelismo.
2. ¹⁰**Indique** quais são os principais desafios que se colocam à exploração do processamento vetorial nos processadores atuais.
3. ¹⁰A memória do tipo HBM (*high-bandwidth memory*), tem vindo a ser apontada como uma solução para o problema da *memory-wall*. **Discuta** até que ponto isto será verdade, dando um especial ênfase no contexto das arquiteturas com um elevado número de PUs (*cores*).
4. ¹⁰**Indique** quais são os principais fatores que impedem que um sistema de computação paralelo atinga o pico de desempenho na execução de aplicações paralelas reais.
5. ²⁰**Discuta** as principais diferenças entre os modelos de programação OpenMP, CUDA e MPI em termos da gestão da localidade dos dados.
6. ¹⁰O modelo de programação CUDA organiza as *threads* em blocos. **Explique** as consequências desta organização termos de implementação dos programas paralelos.

Parte II (65% -130 pontos)

Considere as seguintes funções sobre um vetor de estruturas de dados do tipo *Particle*.

```
#define N 5000

typedef struct Px {
    float dx;
    float vx;
} Particle;

float pot = 0.0;
Particle P[N];
float F[N];

void operation_1() {
    for (int n = 0; n < N-1; n++)
        P[n].dx += P[n].vx + F[n] + P[n+1].dx;
}

void operation_2() {
    for (int n = 0; n < N; n++) {
        P[n].vx += F[n];
    }
}

void operation_3() {
    for (int j = 0; j < N; j++)
        F[j] = (float) 0.0;

    for (int j = 0; j < N; j++) {
        for (int i = j+1; i < N; i++) {
            float rd = pow(P[i].dx-P[j].dx,2);
            pot += ( 1.0 /pow(rd,6) - 1.0 / pow(rd,3) );
            float r74 = 1.0 / pow(rd,7) - 1.0 / pow(rd,4);
            F[i] += P[i].dx * r74;
            F[j] -= P[i].dx * r74;
        }
    }
}

void simulation() {
    // ...
    operation_1();
    operation_2();
    operation_3();
    // ...
}
```

1. ³⁰Pretende-se analisar e otimizar a versão sequencial deste programa.
 - a) ¹⁵**Explique** as vantagens da fusão do procedimento `operation_1` com `operation_2` e indique, justificando, se, nessa fusão, a soma realizada em `operation_2` poderá ser realizada antes das operações de `operation_1`.
 - b) ¹⁵**Identifique, justificando**, os principais obstáculos à vectorização automática (i. é., realizada pelo compilador) de `operation_3`.

2. ⁴⁰**Desenvolva** uma versão paralela do código com **OpenMP**.
 - a) ¹⁰**Discuta, argumentando**, se a função `operation_1` poderá/deverá ser paralelizada.
 - b) ³⁰**Desenvolva** a versão paralela da função `operation_3`, e **comente** o propósito de cada uma das diretivas que usar no contexto desta função.

3. ³⁰Pretende-se desenvolver um *kernel* em CUDA para implementar a `operation_3`. Considere o esboço de um possível *kernel* CUDA apresentado em baixo e ignore as chamadas dos *kernels*, as alocações e as cópias de memória entre *host* e *device*.

```
__global__ void operation_3(Particle *P, float *F, int n, float *pot {  
    int thread_id = threadIdx.x + blockIdx.x * blockDim.x;  
  
    ... //
```

- a) ²⁰**Apresente** o(s) *kernel*(s) CUDA que implementam a `operation_3`. Quantas *threads* deverão ser lançadas? Justifique.
 - b) ¹⁰**Indique, justificando**, que otimizações poderão melhorar o desempenho.
4. ³⁰Pretende-se desenvolver uma versão paralela da função `simulation` para memória distribuída com **passagem de mensagens (i.e., MPI)**, em que um processo principal contém os dados originais e, no final da execução, deverá conter os dados processados. Considere uma solução apenas com 3 processos.
 - a) **Apresente**, através de um diagrama, o que deve ser processado em cada processo, bem como as mensagens trocadas pelos vários processos. Assuma que a função `simulation` é chamada várias vezes.
 - b) **Implemente** um esqueleto do código da solução desenhada na alínea anterior. Apresente um pseudo-código com as primitivas de comunicação identificadas na alínea anterior e as chamadas às várias funções.